1962



125-1-19

INTEGRATED SYSTEMS PROJECT

GENERAL ELECTRIC
MANUFACTURING SERVICES
MATERIALS SERVICE
NEW YORK OFFICE

Integrated Data Store - The Information Processing Machine That We Need!

Presentation Made

at

General Electric Computer Users Symposium
Kiamesha Lake, New York
May 17-18, 1962

by

Charles W. Bachman, Engineer
Integrated Systems Research & Development
Production Control Service
New York, New York

This material is for General Electric employees only:
it may not be reproduced without approval of Production
Control Service.

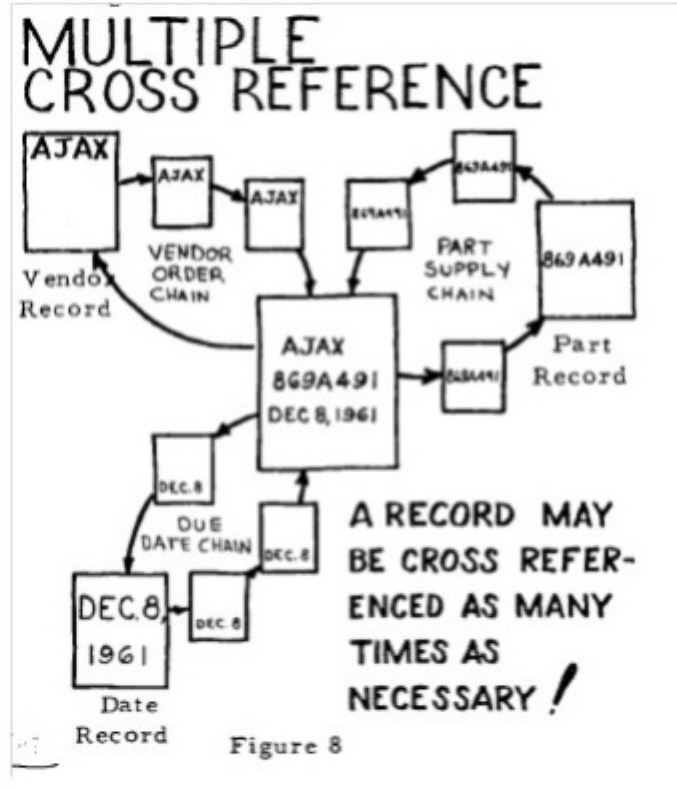


Figure 8

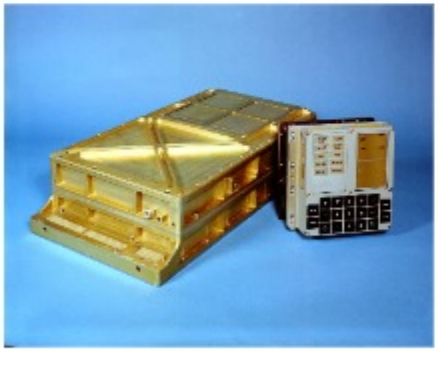
1966


LEGO
NASA APOLLO
SATURN V
14+
92176
1969
pcs/pas
APOLLO
NASA
LEGO IDEAS


LEGO
CREATOR
EXPERT
Age/edades
16+
10266
NASA Apollo 11
Lunar Lander
1087
pcs/pzs
UNITED STATES

1969/70





Information Retrieval — P. BAXENDALE, Editor

A Relational Model of Data for Large Shared Data Banks

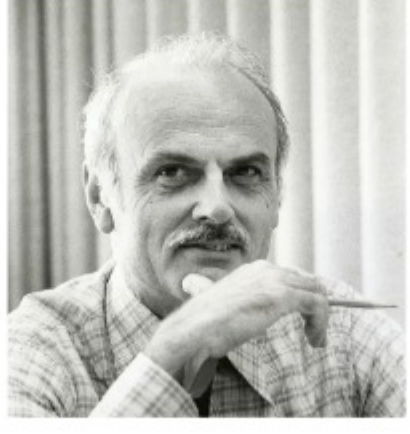
E. F. Codd
IBM Research Laboratory, San Jose, California

Future users of large data banks must be protected from having to know how the data is organized in the machine (the internal representation). A prompting service which supplies such information is not a satisfactory solution. Activities of users at terminals and most application programs should remain unaffected when the internal representation of data is changed and even when some aspects of the external representation are changed. Changes in data representation will often be needed as a result of changes in query, update, and report traffic and natural growth in the types of stored information.

Existing noninferential, formatted data systems provide users with tree-structured files or slightly more general network models of the data. In Section 1, inadequacies of these models are discussed. A model based on n-ary relations, a normal form for data base relations, and the concept of a universal data sublanguage are introduced. In Section 2, certain operations on relations (other than logical inference) are discussed and applied to the problems of redundancy and consistency in the user's model.

KEY WORDS AND PHRASES: data bank, data base, data structure, data organization, hierarchies of data, networks of data, relations, derivability, redundancy, consistency, composition, join, retrieval language, predicate calculus, security, data integrity
CR CATEGORIES: 3.70, 3.73, 3.75, 4.30, 4.22, 4.29

The relational view (or model) of data described in Section 1 appears to be superior in several respects to the graph or network model [3, 4] presently in vogue for noninferential systems. It provides a means of describing data with its natural structure only—that is, without superimposing any additional structure for machine representation purposes. Accordingly, it provides a basis for a high level data language which will yield maximal independence between programs on the one hand and machine representation and organization of data on the other.



3.1 Simple Query

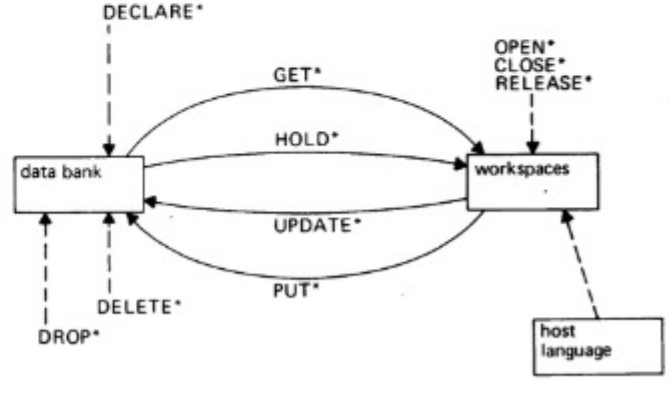
** Find all the part numbers of parts being supplied

```
GET  W  SUPPLY.P#
```

The set of distinct part numbers appearing in the part number attribute (P#) of the relation SUPPLY is copied into workspace W. Note that redundant duplicate values of P# are not delivered to W. After execution of this statement, W may be treated as a unary relation, whose sole attribute is P#.

This query has three principal parts:

an operation name	GET
a workspace name	W
a target list	SUPPLY.P#



1973/74



QUEL	SQL

`create student(name = c10, age = i4, sex = c1, state = c2)`
 	
`range of s is student`
`append to s (name = "philip", age = 17, sex = "m", state = "FL")`
 	
`retrieve (s.all) where s.state = "FL"`
 	
`replace s (age=s.age+1)`
 	
`retrieve (s.all)`
 	
`delete s where s.name="philip"`
	`create table student(name char(10), age int, sex char(1), state char(2));`
	
	`insert into student (name, age, sex, state) values ('philip', 17, 'm', 'FL');`
	
	`select * from student where state = 'FL';`
	
	`update student set age=age+1;`
	
	`select * from student;`
	
	`delete from student where name='philip';`



Michael Stonebraker

INGRES

Q2. Find the average salary of employees in the Shoe Department.

$$\mathrm{AVG}\left(\underset{\mathrm{SAL}}{\mathrm{EMP}'} \underset{\mathrm{DEPT}}{(\text{'SHOE'})}\right)$$

Mappings may be *composed* by applying one mapping to the result of another, as illustrated by Q3.

Q3. Find those items sold by departments on the second floor.

$$\underset{\mathrm{ITEM}}{\overset{\mathrm{SALES}}{\mathrm{DEPT}}} \circ \underset{\mathrm{DEPT}}{\overset{\mathrm{LOC}}{\mathrm{FLOOR}}}(2)$$

The floor '2' is first mapped to the departments located there, and then to the items which they sell. The range of the inner mapping must be compatible with the domain of the outer mapping, but they need not be identical, as illustrated by Q4.

Q4. Find the salary of Anderson's manager.

$$\underset{\mathrm{SAL}}{\overset{\mathrm{EMP}}{\mathrm{NAME}}} \circ \underset{\mathrm{NAME}}{\overset{\mathrm{EMP}}{\mathrm{MGR}}}(\text{'ANDERSON'})$$

History [edit]

SQL was initially developed at IBM by Donald D. Chamberlin and Raymond F. Boyce after learning about the relational model from Edgar F. Codd[13] in the early 1970s.[14] This version, initially called SEQUEL (Structured English Query Language), was designed to manipulate and retrieve data stored in IBM's original quasirelational database management system, System R, which a group at IBM San Jose Research Laboratory had developed during the 1970s.[14]

Chamberlin and Boyce's first attempt at a relational database language was SQUARE (Specifying Queries in A Relational Environment), but it was difficult to use due to subscript/superscript notation. After moving to the San Jose Research Laboratory in 1973, they began work on a sequel to SQUARE.[13] The original name SEQUEL, which is widely regarded as a pun on QUEL, the query language of Ingres,[15] was later changed to SQL (dropping the vowels) because "SEQUEL" was a trademark of the UK-based Hawker Siddeley Dynamics Engineering Limited company.[16] The label SQL later became the acronym for Structured Query Language.[17]

1979



1983

DB2®



1986/87





1986-95

THE DESIGN OF POSTGRES

Michael Stonebraker and Lawrence A. Rowe

Department of Electrical Engineering
and Computer Sciences
University of California
Berkeley, CA 94720



PostgreSQL
the world's most advanced open source database

Abstract

This paper presents the preliminary design of a new database management system, called POSTGRES, that is the successor to the INGRES relational database system. The main design goals of the new system are to:

1) provide better support for complex objects,

2) provide user extendibility for data types, operators and access methods,

3) provide facilities for active databases (i.e., alerters and triggers) and inferencing including forward- and backward-chaining,

4) simplify the DBMS code for crash recovery,

5) produce a design that can take advantage of optical disks, workstations composed of multiple tightly-coupled processors, and custom designed VLSI chips, and

6) make as few changes as possible (preferably none) to the relational model.

The paper describes the query language, programming language interface, system architecture, query processing strategy, and storage system for the new system.

1. INTRODUCTION

The INGRES relational database management system (DBMS) was implemented during 1975-1977 at the Univerisity of California. Since 1978 various prototype extensions have been made to support distributed databases [STON83a], ordered relations [STON83b], abstract data types [STON83c], and QUEL as a data type [STON84a]. In addition, we proposed but never prototyped a new application program interface [STON84b]. The University of California version of INGRES has been "hacked up enough" to make the inclusion of substantial new function extremely difficult. Another problem with continuing to extend the existing system is that many of our proposed ideas would be difficult to integrate into that system because of earlier design decisions. Consequently, we are building a new database system, called POSTGRES (POST inGRES).